

# Agentic Geoscientific Modelling -- Question-Archetype Framework (NATE reference, 2026-06-26)

---

Source: NATE's external-AI conversation, archived here verbatim (below the GRACE-2 bridge note). NATE: "save it so we have a path forward when I resume." Convert to PDF with `pandoc agentic_geo_models_archetype_framework.md -o agentic_geo_models_archetype_framework.pdf`.

## GRACE-2 bridge note (how this maps onto what we already have)

---

This document formalizes a direction GRACE-2 is ALREADY part-way down. Cross-references:

- **Question-archetype template = our composer pattern.** Our `model_*` composers (e.g. `model_flood_scenario`, `model_contamination_affected_fields`, `model_glm_lightning_animation`) are exactly "templates that encode which packages/options to use, what data to fetch, what derived outputs to compute and map." This doc argues for a *library* of them, one per real-world question archetype, declared as data (YAML/JSON) rather than only as Python. See memory `[[project_tool_integration_paradigm]]` (Class A discoverable vs Class B explicitly-defined) and `[[reference_engine_cloud_ai_drivability_ranking]]`.
- **Two-stage router -> domain-specialist** (the last exchange): a cheap interpreter pass classifies the question + extracts params + selects a template, then a domain agent (groundwater/flood/seismic) fills placeholders and generates the run. This is the per-turn routing we DEFERRED in `[[project_tool_retrieval_rag_for_local_models]]` (task #162) and the model-selector/agent-tier work `[[project_model_selector_and_agent_tier]]`. The template is the CONTRACT between router and specialist.
- **Invariant core + differentiating packages.** MODFLOW always needs TDIS/DIS/NPF/STO/OC/IC and yields heads+budget; the archetype only adds the differentiating packages (WEL/RIV/RCH/GWT/BUY/DRN...). Our `gwt_adapter.py` already does GWF+GWT; the archetype list below is the backlog of what ELSE the same engine can answer with a different package set.
- **Engines we have vs. the archetype surface:** SFINCS (coastal/pluvial/fluvial/ compound) + MODFLOW6 GWF/GWT + PySWMM are live (S-tier). OpenQuake is the earthquake-risk archetype family (GEM source/exposure/vulnerability) -- not yet built; it slots into the same data->build->run->map->report decomposition.
- **Mapping-first output.** Every archetype's deliverable is spatial (depth/hazard maps, head+flow, plume contours, PGA/loss maps) -- aligns with our `publish_layer` + `TiTiler` + `deck.gl` rendering spine and the time-series animation track `[[project_timeseries_animation_and_overlay_layout]]`.

### Actionable backlog this seeds (NOT yet scheduled -- await NATE go):

1. Declare existing composers as data-driven archetype manifests (id, model, required/optional packages, required\_inputs, outputs, visualizations).
2. Add a thin interpreter/router pass (archetype-classify + param-extract) in front of the agent, logging the selected template (traceability).
3. Backfill MODFLOW archetypes we can reach cheaply from the GWF/GWT seam: capture-zone (MODPATH), saltwater intrusion (BUY), mine dewatering (DRN), regional budget, MAR. SFINCS

archetypes: pluvial, fluvial, compound, wind-setup, levee-breach, tsunami (mostly forcing/ physics-switch changes on the same exe).

4. OpenQuake as the third engine family (GEM models are pre-packaged + scriptable -- "easy" per the doc).

---

## Full Conversation: Agentic Geoscientific Modelling

---

This document contains every message exchanged in the chat about building an agentic system for groundwater (MODFLOW/FloPy), flood (SFINCS), and earthquake (OpenQuake) modelling, including canonical examples, data needs, mapping, templates, and architecture.

---

### User

---

What is the canonical case study or example of MODFLOW using flopy and how do we get useful data visualizations that answer meaningful questions about ground water dynamics

### Assistant (summary of the canonical example)

---

The canonical FloPy case study is the **Freyberg (1988)** synthetic transient multi-layer benchmark -- a compact model (2 layers, 1x20 grid, 11 transient stress periods, CHD + RIV + WEL + RCH) whose synthetic heads serve as calibration "observations." FloPy ships a complete notebook ("Freyberg: A MODFLOW 6 Example").

Meaningful groundwater-dynamics questions + their visualizations:

| Question                                         | Visualisation                              |
|--------------------------------------------------|--------------------------------------------|
| Where is the water table and how does it change? | Head contour maps / head time series       |
| Direction + speed of flow?                       | Flow vectors, particle pathlines (MODPATH) |
| How much water enters/leaves?                    | Water budget bar/stacked-area charts       |
| Pumping/recharge effect on local heads?          | Hydrographs at observation wells           |
| Confined/unconfined transitions, dry cells?      | Saturated thickness / active-cell maps     |
| River-aquifer interaction?                       | Leakage time series, head cross-section    |

Building blocks (all pure Python via FloPy + matplotlib):

- **Head + flow vectors:** `gwf.output.head()` , `bud.get_data(text='SPDIS')` , `flopy.utils.postprocessing.get_specific_discharge` , `flopy.plot.PlotMapView` .
- **Hydrographs:** `gwf.output.head().get_alldata()` at selected cells.
- **Cross-section:** `flopy.plot.PlotCrossSection(line={'row': 0})` .
- **Water budget:** `gwf.output.budget().get_dataframes()` .
- **Particle tracking:** `flopy.modpath.Modpath7` -> pathlines/endpoints.

Key takeaway: Freyberg is canonical because it is compact yet contains all essential BCs, transient stresses, and observation data; the visualizations turn model numbers into a clear story of how water moves.

## User

---

I want to do this all in python am I loosing functionalities will I be able to answer all of those questions?

## Assistant

---

No loss of functionality. FloPy gives full programmatic access to all MODFLOW input/output; every listed question is answerable in Python (often more flexibly than a GUI). You also GAIN: automation (hundreds of scenarios in a loop), custom metrics, version control, and full transparency. The Freyberg notebook itself builds, runs, post-processes, and even sets up PEST calibration -- all pure Python.

## User

---

Ok now that we have the canonical example of its capabilities what is a real world end to end question I could answer

## Assistant (the real-world question)

---

**"What is the maximum sustainable pumping rate for a proposed municipal wellfield in an alluvial valley, such that after 10 years of operation (including a 2-year severe drought) streamflow depletion in the adjacent ecologically sensitive river never exceeds 20% of the lowest weekly mean flow?"** -- a classic conjunctive-use problem.

End-to-end Python/FloPy workflow:

1. Build the conceptual model from site data (aquifer geometry, K, river stage/conductance, recharge, ET, head observations).
2. Assemble the MF6 model with `flopy.mf6` (NPF, CHD, RCH, EVT, RIV, WEL, STO...).
3. Calibrate to historic data (FloPy PEST / pyEMU interface).
4. Run baseline + drought transient; loop over pumping candidates to find the threshold (update WEL, run, extract RIV leakage, test the 20% constraint).
5. Answer with targeted maps: streamflow-depletion time series, capture-zone mapping (MODPATH), drawdown maps, water-budget stacked area, spatial river leakage.
6. Deliver a defensible management answer, e.g. "sustainable rate = 1,200 m<sup>3</sup>/d; 65% from storage, 30% induced river recharge, 5% captured discharge."

Why Python excels: automation, custom constraints (20% depletion = a few lines of pandas), reproducibility, integration (climate projections, optimization, ML surrogates).

## User

---

What data do I need and is it readily available also how would I visualize this on a map?

## Assistant (data + mapping)

---

Most data is publicly available (US especially), though processing is often needed:

| Category                 | Data                               | Typical public source (US)                                                  |
|--------------------------|------------------------------------|-----------------------------------------------------------------------------|
| Aquifer geometry         | layer top/bottom elevations        | USGS 3DEP (10m DEM), state borehole logs, Nat'l Groundwater Atlas           |
| Hydraulic properties     | K, anisotropy, Sy, Ss              | regional studies, state aquifer DBs, pump-test reports, USGS model archives |
| River geometry/<br>stage | bed elev, stage TS, conductance    | USGS NWIS gauges, NHDPlus, bathymetry surveys                               |
| Recharge                 | net recharge (P - ET - runoff)     | PRISM, MODIS/SSEBop ET, USGS soil-water-balance                             |
| Evapotranspiration       | extinction depth, max ET, land use | NLCD, CIMIS/GridMET, literature                                             |
| Initial/boundary         | initial heads, CHD                 | USGS NWIS water levels, regional models                                     |
| Pumping                  | well locations + historic rates    | state water agencies, USGS water use                                        |
| Observations             | heads, river discharge             | NWIS, state networks                                                        |
| Climate/drought          | recharge-reduction, 7Q2            | USGS StreamStats, NOAA Atlas 14, downscaled projections                     |

Bottom line: a public-data screening model is feasible; K is the least certain (calibrate against observed heads/flows).

Mapping the answer:

- **Static map:** `flopy.plot.PlotMapView` for RIV leakage array (`RdBu_r`), overlay well points (geopandas), river line (shapefile), satellite basemap (contextily).
- **Interactive web map:** folium Choropleth over the grid shapefile + CircleMarker wells -> `depletion_map.html`.

Real-world feasibility: high data availability for managed alluvial basins; model built entirely in Python; maps directly answer "is the river depleted beyond the allowable threshold, and where?"

## User

Let's say I'm building an agentic system, what are the steps and how would I need to break this process up to make the ai create a screening model?

## Assistant (the agentic decomposition)

Agent = LLM (plan/reason/write code) + Python sandbox (flopy, modflow6, numpy, pandas, matplotlib, geopandas, contextily) + data-retrieval tools + a model-state tracker. Stages:

- **Stage 1 - Question interpretation & parameter extraction.** Pure LLM. Emit a structured task dict: `{type, target{metric, max_fraction, duration_years}, drought{length_years, recharge_reduction}, location{lat, lon, extent_km}, outputs[...]}`.
- **Stage 2 - Data acquisition (automated).** Tool calls: DEM (USGS 3DEP/ OpenTopography), aquifer bottom (state DBs / regional assumption), K & storage (literature/regional model), river stage + 7Q2 (NWIS / StreamStats), recharge (GridMET/PRISM), land use (NLCD), wells/ pumping (state data), observations (NWIS). Save raw data + metadata to the workspace.
- **Stage 3 - Model design & code generation.** Write a complete FloPy script: grid, layer elevations, properties, BCs (CHD/RIV/RCH/EVT/WEL), temporal discretization (steady warmup

+ monthly stress periods incl. drought multiplier), OBS points. Use a **code template library** with placeholders + a write/dry-run/ read-raiseback/fix loop.

- **Stage 4 - Calibration (optional).** FloPy PEST interface if observations exist.
- **Stage 5 - Scenario execution.** Loop/binary-search over pumping rates; extract RIV leakage; test constraint; write the max sustainable rate + its time series.
- **Stage 6 - Post-processing & visualisation.** Depletion-vs-time, river-leakage map (+ wells + basemap), drawdown map, water-budget stacked area; annotate the key finding.
- **Stage 7 - Report generation.** Markdown/PDF stitched from data + plots + boilerplate; state assumptions + limitations.

Orchestration = a largely linear state machine (Planner -> stages 1..7). Hard parts: code-gen error handling (retry on traceback), graceful missing-data fallbacks (document them), domain-knowledge gaps (system-prompt cheat sheet + verified templates).

## User

---

Now what would this look like for a model like SFINCS

## Assistant (SFINCS agent)

---

SFINCS (Deltares) = reduced-complexity 2D diffusive-wave / local-inertial flood model on a structured grid; built for rapid compound-flood scenarios.

Canonical end-to-end question: **"Flood extent, depth, hazard for a 1-in-100-year compound event (surge + rainfall) in Charleston SC, current climate vs +0.5 m SLR."**

Data + availability:

- Topo/bathy (continuous sea->land): GEBCO, SRTM, USGS 3DEP, NOAA Coastal DEMs -- scriptable (e.g. `dem_stitcher`).
- Water-level boundary forcing: GTSM-ERA5 tide+surge, ADCIRC catalogues, FEMA/USACE storm DBs; USGS/NOAA river gauges (pyTMD for tides).
- Rainfall: ERA5, IMERG/GPM, Stage-IV, NOAA Atlas 14 design storms (CDS API).
- River discharge: NWIS, GloFAS.
- Land cover -> Manning's n: ESA WorldCover, NLCD, Copernicus.
- Infiltration/soil (Green-Ampt): SoilGrids, SSURGO.

Pipeline mirrors MODFLOW: interpret -> fetch -> build ( `sfincs / SfincsModel : dep , manning , bzs / bnd_type , precip , infiltration , tstop , output_times , write() , run()`  ) -> scenario loop (baseline + SLR) -> post-process the NetCDF ( `sfincs_map.nc : zsmax , uvmax ; hazard = depth x (vel+0.5)` ) -> map-centric answer + report. Differences from MODFLOW: finer grids (10-100 m) but fast runs (more scenarios), time-varying forcing more readily available, even tighter Python integration, more intuitive map output.

## User

---

Most of the visualization should be able to be mapped include this in your analysis, and then let's move too: openquake

## Assistant (OpenQuake agent)

---

SFINCS output is inherently spatial -- every meaningful viz is a map (depth, extent, hazard depth x velocity, exposure overlay, scenario side-by-side, folium web map). Map-centric by nature.

OpenQuake (GEM) = open engine for probabilistic seismic hazard + risk. Canonical question: **"PGA with 10% probability of exceedance in 50 years for Kathmandu Valley, plus building damage losses (repair cost + casualties) for current stock vs a code-compliant scenario."**

Data (GEM makes this remarkably automatable):

- Seismic source model: GEM Global Active Faults, GEM hazard input XML, USGS, EFEHR -- pre-made OpenQuake-format files.
- GMPEs: OpenQuake HMTK built-ins, selected by tectonic region (logic tree).
- Site conditions: global Vs30 (USGS/GFZ) or inferred from DEM.
- Exposure: GEM Global Exposure Model / GED4GEM, OSM footprints.
- Vulnerability: GEM Vulnerability DB fragility curves (pre-packaged).
- Population: WorldPop, GHSL (casualty estimation).

Pipeline: interpret -> fetch (download\_gem\_source\_model, download\_vs30, download\_exposure, get\_vulnerability\_functions, get\_population\_grid) -> build ( openquake.engine / hmtk / commonlib; job.ini + logic trees; or adapt GEM demo models) -> scenario (baseline vs code-compliant vulnerability) -> map outputs (PGA hazard map, average-annual-loss map, per-building damage-state map, folium layers) -> report. Every output is a map (hazard, loss, damage, shakemap, Vs30).

Summary table:

| Feature          | MODFLOW                    | SFINCS                      | OpenQuake                      |
|------------------|----------------------------|-----------------------------|--------------------------------|
| Library          | flopy                      | sfincs                      | openquake.engine, hmtk         |
| Data             | USGS, state DBs, NLCD      | GEBCO, ERA5, ESA WorldCover | GEM source/exposure, USGS Vs30 |
| Outputs          | heads, drawdown, depletion | flood depth/extent/hazard   | PGA/loss/damage maps           |
| Viz              | maps, cross-sections, TS   | flood/hazard maps           | hazard/risk/loss maps          |
| Agent difficulty | moderate (patchy data)     | easier (abundant data)      | easy (GEM ready-made)          |

Same decomposition for all three: interpret, acquire, build-in-code, run scenarios, map, report. No functionality lost; reproducibility + scale gained.

## User

---

Can you now write me a downloadable markdown file containing these reports including canonical use for SFINCS, MODFLOW, and OpenQuake, and the other details we discussed for making this system

## Assistant

[Delivered the consolidated Markdown report -- Introduction; MODFLOW (canonical, real-world, data, viz, agent pipeline); SFINCS (same); OpenQuake (same); the Generic Agentic Architecture. Content identical to the per-domain sections above.]

## User

In the ground water what are the invariants to run the model? Is there only one output from the MODFLOW tool ... or can we answer other different questions ... the breadth of the tool ...

## Assistant (invariants + breadth)

**Invariants (every MODFLOW run needs):** spatial discretization (grid), temporal discretization (stress periods/time steps), hydraulic properties (K, Ss, Sy),

=1 boundary condition, initial conditions (heads), output control. **Raw outputs are always the same type:** head at every cell/timestep + budget (volumetric flows per boundary + storage). Breadth comes from the *setup* + *post-processing*:

| Question type          | What you change/add                   | Derived output                    |
|------------------------|---------------------------------------|-----------------------------------|
| Sustainable yield      | WEL, RIV, transient RCH               | depletion TS, capture zones       |
| Capture-zone analysis  | MODPATH on the flow field             | pathlines, travel times           |
| Contaminant transport  | MT3DMS or MF6 GWT (solute)            | plumes, breakthrough curves       |
| Saltwater intrusion    | SEAWAT or BUY (variable density)      | fresh-salt interface, toe         |
| Mine dewatering        | DRN/GHB (pit), WEL                    | drawdown cone, pit inflow         |
| Wetland hydroperiod    | RCH, EVT, stream stage, fine dt       | water-table duration curves       |
| Regional water balance | large grid, RCH, EVT, RIV             | recharge/discharge maps, baseflow |
| ASR                    | injection/extraction wells, transport | recovery efficiency               |

Multiple canonical benchmarks exist beyond Freyberg (Henry = saltwater intrusion; Hantush = leaky aquifer; USGS TWRI examples; Borden/Cape Cod tracer tests). For an agent, each benchmark becomes a TEMPLATE for a class of question. Implication: the invariant core guarantees the run; a **template library** guarantees the output is meaningful for the specific question. The agent's decision tree: interpret -> classify into one of N templates -> select packages -> assemble data -> build+run -> post-process per template -> map + report. Not a single-purpose Freyberg re-runner.

## User

Are those packages available in python? Build me a framework ... exhaustive list ... templates ... after that ... more of these templates for SFINCS also?



## Assistant -- MODFLOW 6 Question-Archetype Templates (all via

`flopy.mf6` )

---

Invariant core assumed in every template: TDIS, DIS/GDIS, NPF, STO, OC, IC. Only differentiating packages listed.

1. **Sustainable Yield / Conjunctive Use** -- max pumping w/o unacceptable streamflow depletion. Packages: WEL, RIV (or SFR), RCH, EVT(opt); MODPATH for capture zones. Inputs: well schedules, river stage/conductance, recharge TS, ET params, low-flow targets. Outputs: river-leakage TS, drawdown, capture zones, budget. Maps: head+flow vectors, depletion map, pathlines.
2. **Capture-Zone Delineation (Wellhead Protection)** -- time-related capture zone. Packages: WEL, RCH(opt), RIV/CHD (regional flow); MODPATH backward tracking. Outputs: pathlines, 1/5/10-yr travel-time zones. Maps: pathlines on head map, capture-zone polygons.
3. **Contaminant Transport (Point Source)** -- concentration at a receptor after a leak. Packages: WEL(injection), ADV, DSP, SSM (GWT) or MT3DMS. Inputs: source conc/mass, porosity, dispersivity, sorption. Outputs: conc TS, plume snapshots. Maps: conc contours over time, breakthrough curves.
4. **Saltwater Intrusion (Coastal Aquifer)** -- inland toe migration. Packages: BUY (MF6) or SEAWAT ( `flopy.seawat` ). Inputs: freshwater head, seawater boundary conc, density ratio. Outputs: salt distribution, toe (2.5% isochlor). Maps: salinity cross-section, plan-view toe.
5. **Mine Dewatering** -- drawdown + inflow to an open pit. Packages: DRN (pit lake), WEL (dewatering), RCH, EVT. Outputs: drawdown, pit inflow, water-table change. Maps: drawdown contours, affected area.
6. **Wetland Hydroperiod Assessment** -- withdrawals altering inundation freq/duration. Packages: RIV/DRN/GHB (wetlands), RCH, EVT, WEL. Outputs: water-table TS at wetland cells, exceedance curves. Maps: hydroperiod-change.
7. **Regional Groundwater Budget** -- renewable resource + sustainable abstraction. Packages: RCH, EVT, RIV, WEL, CHD. Outputs: steady/transient budget components. Maps: recharge-discharge zones, net flux.
8. **Aquifer Storage and Recovery (ASR)** -- recovery efficiency of injected freshwater in brackish aquifer. Packages: WEL (inject/extract), BUY or GWT tracer. Outputs: conc breakthrough, recovery efficiency. Maps: plume evolution.
9. **Parameter Estimation / Calibration** -- best-fit K + recharge vs observed heads. Packages: any (typ. RCH/WEL/RIV); PEST or `flopy.utils.PestSetup` . Outputs: optimized params, residual maps, uncertainty. Maps: parameter, residual.
10. **Managed Aquifer Recharge (MAR)** -- where/how much to infiltrate. Packages: RCH(enhanced), WEL(extraction), DRN(ponding). Outputs: mounding, recovery rate, waterlogging risk. Maps: mounding, water-table depth. Note: add MODPATH for advective transport, or swap SFR for RIV for detailed streams.

## Assistant -- SFINCS Question-Archetype Templates (physics switches + forcing types)

---

No "packages"; physics switches + forcing types in `sfincs.inp` / Python.

1. **Coastal Surge Flooding** -- inundation from 1-in-100-yr storm tide. Physics: advection=0 (diffusive), manning=, baroclinic=0. BC: bndtype=waterlevel (offshore surge TS). Inputs: DEM/bathy, Manning's n, surge TS. Outputs: max depth/velocity, hazard. Maps: depth, hazard.



2. **Pluvial (Rainfall) Flooding** -- urban flooding from extreme rainfall. Physics: diffusion, precip=1, infiltration=1(opt). BC: closed except outflow. Forcing: gridded precip (mm/h). Outputs: max ponding, street discharge. Maps: depth on street network.
3. **Fluvial (River) Flooding** -- 100-yr river flood extent. Physics: diffusion. BC: bndtype=discharge (upstream hydrograph) + downstream waterlevel/friction. Outputs: extent, water-level profiles. Maps: inundation along river.
4. **Compound Flooding (Surge + Rainfall + River)** -- coincident drivers. Physics: diffusion, precip+infiltration on. BC: waterlevel(offshore) + discharge(upstream) simultaneously. Forcing: bzs/bhs + precip. Outputs: combined depth, hazard. Maps: combined + single-driver comparison.
5. **Infiltration-Sensitive Flooding** -- soil saturation effect on pluvial. Physics: infiltration=1, ks,at, suction, soil\_depth. Inputs: soil maps, K, suction. Outputs: infiltration loss, reduced ponding. Maps: depth-difference vs no-infiltration.
6. **Wind-Driven Flooding / Wave Setup** -- local wind setup. Physics: advection=1 (local inertial, required for wind), wind=1, Cd. BC: offshore waterlevel + wind field. Outputs: water level with setup. Maps: with/without-wind difference.
7. **Dam-Break / Levee-Breach Overland Flow**. Physics: advection=1 (inertial). BC: internal discharge source (breach). Inputs: DEM, Manning's n, breach hydrograph. Outputs: arrival time, max depth. Maps: arrival-time (hazard), max depth.
8. **Tsunami Inundation** -- run-up from a wave height. Physics: advection=1, baroclinic=0(opt). BC: bndtype=waterlevel (tsunami wave TS). Inputs: hi-res DEM, Manning's n, wave form. Outputs: run-up, inundation line. Maps: max inundation on elevation.

## Framework implementation suggestion (YAML template library)

```
id: sustainable_yield
model: modflow6
description: Maximum pumping without exceeding streamflow depletion target
required_packages: [WEL, RIV, RCH]
optional_packages: [EVT, MODPATH]
required_inputs: [dem, river_stage, recharge_raster, pumping_wells]
outputs: [river_leakage_ts, drawdown_map, capture_zone_map]
visualizations: [head_flow_vector_map, depletion_map_basemap, water_budget_stacked]
```

```
id: compound_flood
model: sfincs
physics: {advection: 0, infiltration: true, precipitation: true}
boundary_types: [waterlevel, discharge]
forcings: [bzs, precip, bhs]
outputs: [depth_max, hazard]
visualizations: [flood_depth_map, hazard_map, comparison_sidebyside]
```

The agent selects the best-matching template, populates placeholders from the acquisition step, and writes the flopy/sfincs Python.

## User

Small question but would I need a prompt interpreter pass to then route to a template and from there a groundwater specific agent takes over with the refined interpreted prompt,

## Assistant (two-stage architecture -- the routing decision)

---

Yes. A two-stage process is robust:

1. **Interpreter / Router** -- lightweight LLM pass: classify the question, extract key params (location, scenario, metrics), select the best-matching template.
2. **Domain-specific agent** (Groundwater / Flood / Earthquake) -- receives the refined prompt (template id + extracted params) and takes over. Its system prompt knows the invariant structure, the template's packages/physics, the domain library (flopy/sfincs/openquake), and how to map outputs.

The **template is the contract** between router and specialist; the specialist just fills placeholders (bbox, forcing paths, scenario list) and generates the script. Benefits: clean separation of concerns; safety + cost (cheap router, powerful specialist); scalability (add a domain by adding a specialist + templates, no router change); traceability (template selection is logged). Not strictly required (one agent could do it all) but much cleaner for production.

## User

---

Can you now turn this entire chat into a downloadable pdf?

## Assistant

---

[Provided this Markdown export + pandoc/browser/Typora conversion instructions.]